

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ЛЬВІВСЬКА ПОЛІТЕХНІКА



**АВТОМАТИЗОВАНЕ ПРОЕКТУВАННЯ
КОМП'ЮТЕРНИХ СИСТЕМ**

Завдання 2: “Комунікація клієнта та сервера”

Виконав:

ст. гр. КІ-404

Голодняк А. Р.

Прийняв:

Федак П. Р.

Львів – 2024

ЗАВДАННЯ

1. Створити репозиторій на GitHub з назвою "csad", де основна гілка - develop.

YY1 – рік початку навчання

YY2 – рік закінчення навчання

Наприклад: 2019-2020 -> 1920

Повний приклад: csad1920ki47spitzeras03

2. Додати доступ для автора.

3. Створити гілку feature/develop/.

Наприклад, feature/develop/task1.

4. Створити файл README, де вказати:

a. деталі про репозиторій;

b. деталі завдання;

c. номер студента та інформацію.

d. додати інформацію про технології, мову програмування та обладнання, яке буде використовуватися в наступних завданнях.

5. Створити GIT TAG:

__WW · YY – поточний рік; WW – робочий тиждень; D – номер дня тижня.

6. Створити pull request з назвою task1 та надати викладача як рецензента.

7. Після затвердження рецензента – виконати злиття запиту в гілку develop.

Критерії прийняття - Посилання на створений репозиторій з гілкою "develop" як основною та файлом README.MD.

ТЕОРЕТИЧНИЙ МАТЕРІАЛ

GitHub Actions – це сервіс для автоматизації робочих процесів безпосередньо в GitHub. Він дозволяє налаштовувати CI/CD, автоматичне тестування, деплой та інші процеси для проєктів, використовуючи файли конфігурації на основі YAML.

CMake – це кросплатформений інструмент автоматизації збірки, який використовують для генерації конфігураційних файлів (наприклад, Makefile, Ninja) для компіляції програм. Його перевага в тому, що він дозволяє легко створювати збірки на різних системах.

UART (Universal Asynchronous Receiver-Transmitter) – це апаратний модуль або протокол, що використовується для асинхронної серійної передачі даних між пристроями. Він забезпечує обмін даними через два дроти: один для передачі (TX) і один для прийому (RX).

CI/CD (Continuous Integration / Continuous Delivery) – це підхід до розробки програмного забезпечення, що передбачає автоматизацію процесів інтеграції коду, тестування та доставки нових версій програм. Він допомагає виявляти та виправляти помилки на ранніх етапах розробки.

ВИКОНАННЯ РОБОТИ

- Створіть просту схему комунікації ПЗ (клієнт) <-> UART <-> Апаратне забезпечення (сервер).

```
andrii@andrii-VirtualBox task tree .
├── clientSide
│   ├── CMakeLists.txt
│   ├── include
│   │   └── constants.h
│   └── src
│       └── clientSide.cpp
├── README.md
├── reports
│   └── АПКС_КІ-404_Голодняк_task1.docx
└── serverSide
    ├── CMakeLists.txt
    ├── include
    │   └── constants.h
    └── serverSide.ino
```

- Клієнт повинен відправити повідомлення серверу. Сервер має модифікувати повідомлення і надіслати його назад клієнту.

```
andrii@andrii-VirtualBox build ./client
Opening port '/dev/ttyUSB0' Waiting for Arduino to be ready...
Received from Arduino: Arduino ready!

Enter message to send to Arduino: this is the test :P
Message sent: this is the test :P
Response from Arduino: THIS IS THE TEST :P

Enter message to send to Arduino:
```

- Створіть YML файл з наступними функціями: а. збірка всіх бінарних файлів (створіть скрипти у папці ci/ за потреби);

```
name: CI Pipeline
on:
  push:
    branches:
      - '*'
    pull_request:
      branches:
        - '*'
  jobs:
    build-client:
      runs-on: ubuntu-latest
      steps:
        - name: Checkout repository
          uses: actions/checkout@v3
        - name: Install dependencies
          run: sudo apt-get update && sudo apt-get install -y libserialport-dev
        - name: Setup CMake
          uses: lukka/get-cmake@v3.23.0
        - name: Create build directory
          run: mkdir -p build
```

```

name: Configure CMake          run: |          cd build
    cmake ../clientSide

name: Build client using CMake          run: |          cd build          cmake --build .

#- name: Run tests
# run: |
# TBD          build-server:          runs-on: ubuntu-latest
steps:
name: Checkout repository          uses: actions/checkout@v3

name: Set up Arduino CLI          uses: arduino/setup-arduino-cli@v1          with:          version: '0.19.2'

name: Install Arduino core for AVR boards          run: arduino-cli core install arduino:avr

name: Build Arduino sketch (Server)
  run: arduino-cli compile --fqbn arduino:avr:uno serverSide/serverSide.ino
#dummy
name: Upload Arduino Build Artifact          uses: actions/upload-artifact@v4          with:          name: arduino

#- name: Run tests
# run: |
# TBD

```

ВИСНОВОК

У процесі виконання завдання було створено схему комунікації між програмним забезпеченням (клієнт) і апаратним забезпеченням (сервер) через інтерфейс UART. Клієнт ініціює передачу повідомлення до сервера, який обробляє отримані дані, вносить зміни та надсилає їх назад клієнту, забезпечуючи двосторонню комунікацію.

Крім цього, була створена автоматизована система для збірки та тестування програмного коду, яка включає YAML-конфігурацію для CI/CD процесу. Ця конфігурація автоматизує збірку всіх необхідних бінарних файлів, організацію скриптів у відповідній папці, а також створення артефактів, що містять зібрані бінарні файли та результати тестування. Це дозволяє підтримувати стабільність і надійність розробки через безперервну інтеграцію та доставку.